

README

_tools/containers — this umbrella's containerised workloads

This directory is the umbrella root's **consumer** of the canonical Containers Submodule, vasic-digital/containers (Go module digital.vasic.containers), which is pinned in this tree at submodules/containers.

It is the first and, at this writing, the **only** consumer of that submodule in this repository. Measured 2026-09-03: before this directory existed, `grep -rn 'digital.vasic.containers'` outside submodules/ and workshop/ returned **zero** hits — the gitlink was declared, manifest-pinned and unused.

What is here

Path	What it is
go.mod	consumer module, replace digital.vasic.containers => ../../submodules/containers (§11.4.76(2))
cmd/site-build/	the orchestrator. Drives podman/docker only through the submodule's pkg/runtime, pkg/compose and pkg/logging
cmd/distribute-helixtranslate/	builds the HelixTranslate image on a remote host and distributes it, through the submodule's pkg/remote and pkg/remoteexec. Never run against a live remote — see its package comment before trusting it
cmd/runtime-probe/	

Path	What it is
	answers “which container runtime is on THIS machine?” via the submodule’s runtime.AutoDetect, for shell callers that must not grow their own detection. Three-valued; fully verified on this host
compose/compose.sites.yml	the service definitions the orchestrator consumes
compose/jekyll-build.sh	the Jekyll service’s entrypoint, executed inside the container only

podman and docker appear nowhere in cmd/site-build. That is the point of §11.4.76(4): the submodule owns every process that talks to a runtime, and this tree owns none of them.

Running it

```
cd _tools/containers
go build -o bin/site-build ./cmd/site-build

./bin/site-build -probe                # which runtime + compose
                                     are here
./bin/site-build -workload jekyll      # rebuild milosvasic.ru/
                                     _site
./bin/site-build -workload gen-test    # the generator's unit
                                     tests, pinned toolchain

go build -o bin/runtime-probe ./cmd/runtime-probe
./bin/runtime-probe                    # runtime=podman
                                     version=5.7.1
./bin/runtime-probe -name-only          # podman                (for `$(
                                     (...)` in shell)
```

runtime-probe exists because site-build -probe cannot answer for a caller that has no compose file: it returns **2** when compose is unavailable, which is correct for a compose orchestrator and wrong for the question “is there a runtime here at all”. runtime-probe asks only runtime.AutoDetect.

Verified on this host, all three exit codes against real conditions (2026-09-04, podman 5.7.1):

```
./bin/runtime-probe                runtime=podman
version=5.7.1                      rc 0
./bin/runtime-probe extra          unexpected argument
"extra"                            rc 1
```

```

env PATH=<empty dir> ./bin/runtime-probe  COULD NOT DETERMINE — no
container

runtime is available

here: tried

podman, docker, nerdctl,
cri-o,
lxd,
kubernetes rc 2

```

The rc-2 leg is the paired §1.1 mutation and it is **DATA** — an empty PATH — so it cannot be made inoperative by editing the code it guards. It is asserted again in cmd/runtime-probe/main_test.go; go test ./... is green across the module.

_tools/gen/build.sh builds and calls the jekyll workload itself; VASIC_JEKYLL_MODE (container | host | auto, default auto) picks the strategy there.

Build the binary; do not go run it. go run collapses every non-zero program exit into 1 and prints exit status N instead. Measured 2026-09-03, same code, same arguments:

```

go run ./cmd/site-build -root /tmp -probe -> rc 1 (wrong)
./bin/site-build -root /tmp -probe -> rc 2 (COULD NOT DETERMINE)

```

Exit codes are three-valued, as every check in this tree is: **0** the workload ran and its output was verified, **1** a real failure, **2** COULD NOT DETERMINE — no runtime, no compose command, a compose file that cannot be read, or an outcome that could not be read back. **A 2 is never a pass**, and a 2 arriving as a 1 is an unproven claim reported as a defect.

The freshness assertion is proved, not asserted (§1.1)

The check that makes a green run mean something is “*the artifact was rewritten by THIS run*”. It was exercised against a paired mutation — same service name, same container name, a container that exits 0 and writes nothing:

```

services:
  jekyll-build:
    image: docker.io/library/alpine:3.20
    container_name: vasic-jekyll-build
    command: ["true"]

```

```

site-build: FAIL — milosvasic.ru/_site/index.html was NOT
rewritten:
mtime is still 2026-09-03T20:24:29+02:00. The container exited 0
while

```

```
leaving the previous artifact in place.  
rc 1
```

The mutation is DATA — a throwaway compose file — so the assertion cannot be made inoperative by editing the code it guards.

Why the Jekyll service exists — the measured defect

On the development host, `bundle exec jekyll` exits **127** and `bundle install` **cannot** fix it:

```
mkmf.rb can't find header files for ruby at /usr/lib/include/  
ruby.h
```

```
An error occurred while installing json (2.9.1), and Bundler  
cannot continue.
```

`ruby-3.3.8-alt3` and `libruby-3.3.8-alt3` are installed; **libruby-devel** **is not**, so no gem with a C extension can build. Installing it is a root-level operator action (`sudo` here requires a password).

The consequence was quiet rather than loud. `_tools/gen/build.sh` called a bare `jekyll`, `set -e` aborted the script at that line, `_site` was never rewritten — and `_tests/playwright.config.js` serves exactly that directory. Gate 6 went on reporting a green suite while validating a **six-day-old** artifact for `milosvasic.ru` (`_site/index.html` was dated 2026-08-28 when this was measured on 2026-09-03).

The container carries its own headers and toolchain, so the build needs no host privilege at all — on this machine, and on every future clone.

Why there is no Containerfile here

Both services run a **stock upstream image** with a bind-mounted entrypoint. Nothing about either workload needs a derived image, so none is built. If one ever is needed, §11.4.76(4) says extend `vasic-digital/containers` — its `pkg/crossbuild` already ships Containerfiles behind a Backend seam — rather than growing a parallel image-build path in this tree.

Known upstream defect in the consumed submodule

REPORTED UPSTREAM 2026-09-03 as [vasic-digital/containers#2](#) (open). The issue carries the CLI reproduction below plus a Go program that drives the defect through the module's own public API, and the one-line fix. Re-measured before filing, against gitlink d940b51fc247c285c805799452992da8d09c75b9 on podman 5.7.1 / Go 1.26.2. Filing it is not fixing it — this section stays until the submodule ships the change and the gitlink is bumped.

Measured while filing, and worth recording here: podman.go is the **only** runtime in the package that forwards the "all" sentinel to a backend that cannot parse it. [crio.go:304-308](#) and [lxd.go:328-332](#) guard it with `strconv.Atoi`; [kubernetes.go:418-420](#) guards it with an explicit `o.Tail != "all"`. The guard podman.go is missing already exists three times in its own package.

`runtime.Logs()` returns **zero bytes and a nil error** against the podman runtime when called with default options.

- [pkg/runtime/options.go:141](#) — `defaultLogOptions()` sets `Tail: "all"`.
- [pkg/runtime/podman.go:331-333](#) — that value is appended verbatim as `--tail all`.
- Podman parses `--tail` with `strconv.ParseInt`. Measured on this host:

```
podman logs --tail all <c>    Error: invalid argument "all" for
"--tail" flag ... rc 125
podman logs --tail -1 <c>    <the container's
output>                      rc 0
```

The failure is **silent**: `ExecuteStream` starts the process successfully, so `Logs()` returns a nil error; the pipe hits EOF immediately; the rc-125 reaches the caller only through `Close()` -> `cmd.Wait()`. A caller that defers `Close()` and ignores its error sees *"logs read fine, container printed nothing"* — the exact shape of a bluff. It was observed here first as two green runs with empty log sections.

The fix belongs upstream (§11.4.76(4)): make the podman path translate "all" to "-1", or default to "-1". Nothing in this tree works around it by shelling out to a runtime. What `cmd/site-build` does instead is (a) use the module's own public `runtime.WithTail("-1")`, and (b) report `Close()`'s error instead of discarding it, so a broken log call can never again be mistaken for a quiet container.

Not containerised, and why

- **Playwright (gate 6)** — it works on the host today, so it fixes nothing broken; its webServer binds host TCP ports and serves two directories; and the image it would need is not on this host (podman images lists no playwright image), so it is a multi-GB pull for no measured gain.
- **The page generator itself** (gen, as opposed to its tests) — writes into two site submodules and syncs design-system/ by host path. A candidate, not done, and named here rather than left implicit.
- **workshop-curriculum_platform_1** — already containerised and deliberately untouched.
- **_tools/helixtranslate-container*** — a pre-existing §11.4.76(4) finding, not a gap this directory created. _tools/distribute-helixtranslate.sh builds and replicates an image with raw ssh + podman build + podman save | docker load, and _tools/helixtranslate-container.sh runs it over ssh; submodules/containers/cmd/deploy-stack plus pkg/remote / pkg/distribution exist to do exactly that. It was **not** converted here because it targets remote hosts this session could neither reach nor test, and converting it unverified would be bluff work.

The HelixTranslate surface, re-measured 2026-09-04

The line above stays true for the REMOTE half. The surface was then measured file by file, and it splits three ways rather than two — the middle column is the one an “it’s all a violation” reading loses:

File	§11.4.76 verdict	State
_tools/helixtranslate-container/Containerfile	not a violation. The anchor forbids reimplementing runtime/compose/lifecycle primitives; an image recipe is none of those, and the module itself ships *.Containerfile under pkg/crossbuild	left as is
_tools/helixtranslate-container/Containerfile.translator	same	left as is

File	§11.4.76 verdict	State
_tools/helixtranslate-local.sh	real violation. Froze the literal podman for the LOCAL host — the module owns local runtime detection (runtime.AutoDetect)	CONVERTED to cmd/runtime-probe, verified on this host
_tools/distribute-helixtranslate.sh	real violation. Raw ssh/scp/rsync + podman build + save load — pkg/remote, pkg/remotexec, pkg/distribution own all of it	replacement written, UNVERIFIED (hosts unreachable); original kept
_tools/helixtranslate-container.sh	real violation (raw ssh), not closeable today — it streams the document on remote STDIN and no executed module API accepts stdin	declared exception in its own header
_tools/helixtranslate-container/run.sh	not closeable today — one-shot run --rm -i with stdin; takes its runtime from \$1, so it detects nothing	declared exception added
_tools/translate-fleet.sh	not a violation. Its host == amber.local ? docker : podman is per-host CONFIGURATION: pkg/remote.RemoteHost carries a declared Runtime string field and the module ships no remote runtime detector	note added, code unchanged

The single upstream change that would close the two “not closeable” rows is an **ephemeral-run primitive that accepts stdin** in pkg/runtime — §11.4.76(4) says add it to vasic-digital/containers, never as a parallel implementation here. Measured at gitlink d940b51fc247c285c805799452992da8d09c75b9: ContainerRuntime declares Name, Version, IsAvailable, Start, Stop, Remove, Status, List, Stats, Exec and Logs — **no Run, no Create** — Exec takes no

stdin, and the only WithStdin in the module (pkg/remote/connection/interface.go:146) sits in an interfaces-and-options-only package that nothing implements and no constructor returns.

Unverified end to end, and labelled so here as well as in the file: no helixtranslate:cli image exists on this host (podman images matches it zero times), so the converted helixtranslate-local.sh was exercised for runtime resolution and exit codes only — never for an actual translation. Its rc-1 “translation failed” on this host is the image being absent, and is the same result the pre-conversion script produced, so nothing was downgraded.